

## ✓ Visualization Library Documentation

### Matplotlib and Seaborn

Beginner Level - ShadowFox Data Science Internship

## 1. Library Overview

### Matplotlib

Matplotlib is a Python library used to create static and customizable data visuALIZATIONS. It is useful for exploring data, presenting trends, and creating graphs for reports.

### Seaborn

Seaborn is a Python visualization library built on Matplotlib. It provides a simpler interface for creating attractive statistical graphics and works well pandas DataFrames.

### Why these Libraries?

Matplotlib provides detailed control over graphs, while Seaborn makes statistical visualizations easier to create. Together, they cover many common data visualization requirements.

## ✓ 2. Matplotlib - Common Graph Types

### 2.1 Line Plot

A line plot connects data points to show changes or trends over a sequence.

**Use case:** Comparing monthly sales or tracking temperature over time.

### 2.2 Bar Chart

A bar chart uses rectangular bars to compare values across categories.

**Use case:** Comparing sales across different products.

### 2.3 Scatter Plot

A scatter plot displays individual points to show the relationship between two numerical variables.

**Use case:** Studying the relationship between study hours and marks.

## 2.4 Histogram

A histogram groups numerical values into intervals to show their distribution.

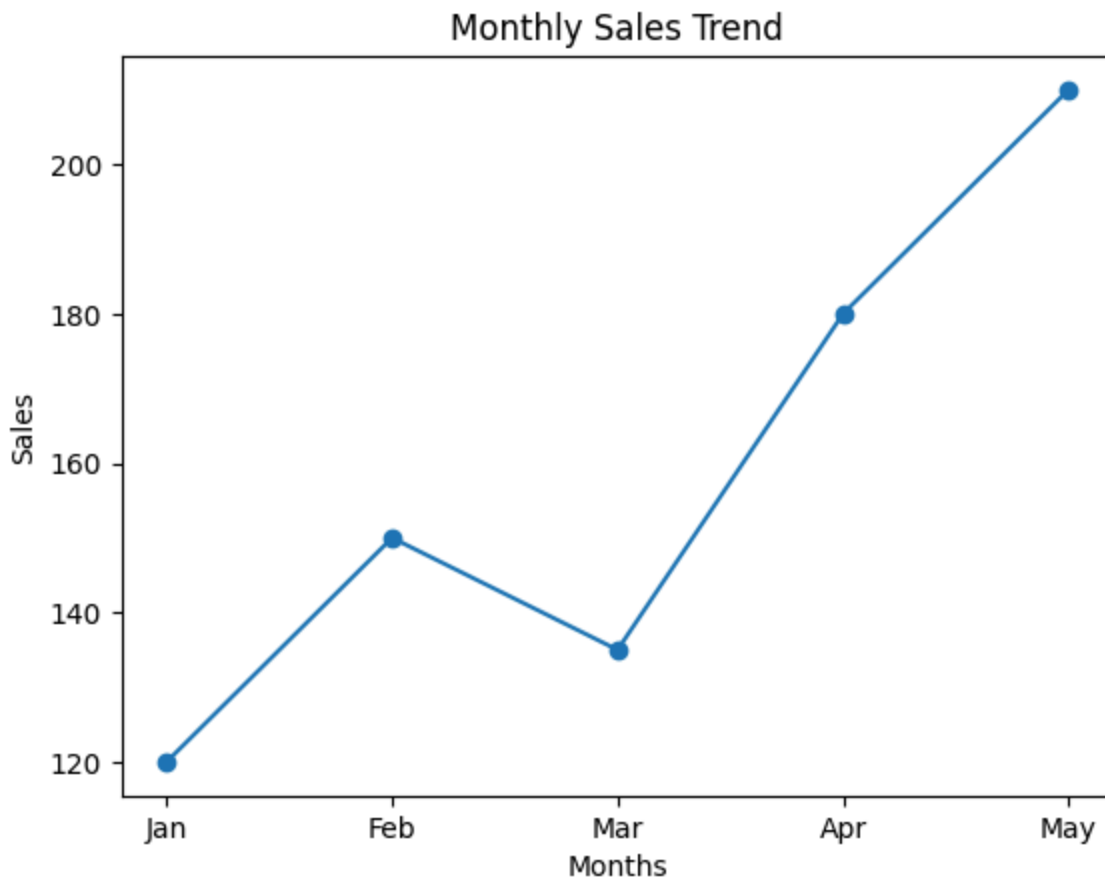
**Use case:** Understanding the distribution of students' marks.

## 2.5 Pie Chart

A pie chart represents parts of a whole using different slices.

**Use case:** Showing the percentage contribution of different categories.

```
1 import matplotlib.pyplot as plt
2
3 months=["Jan","Feb","Mar","Apr","May"]
4 Sales=[120,150,135,180,210]
5
6 plt.plot(months,Sales,marker="o")
7 plt.title("Monthly Sales Trend")
8 plt.xlabel("Months")
9 plt.ylabel("Sales")
10 plt.show()
```

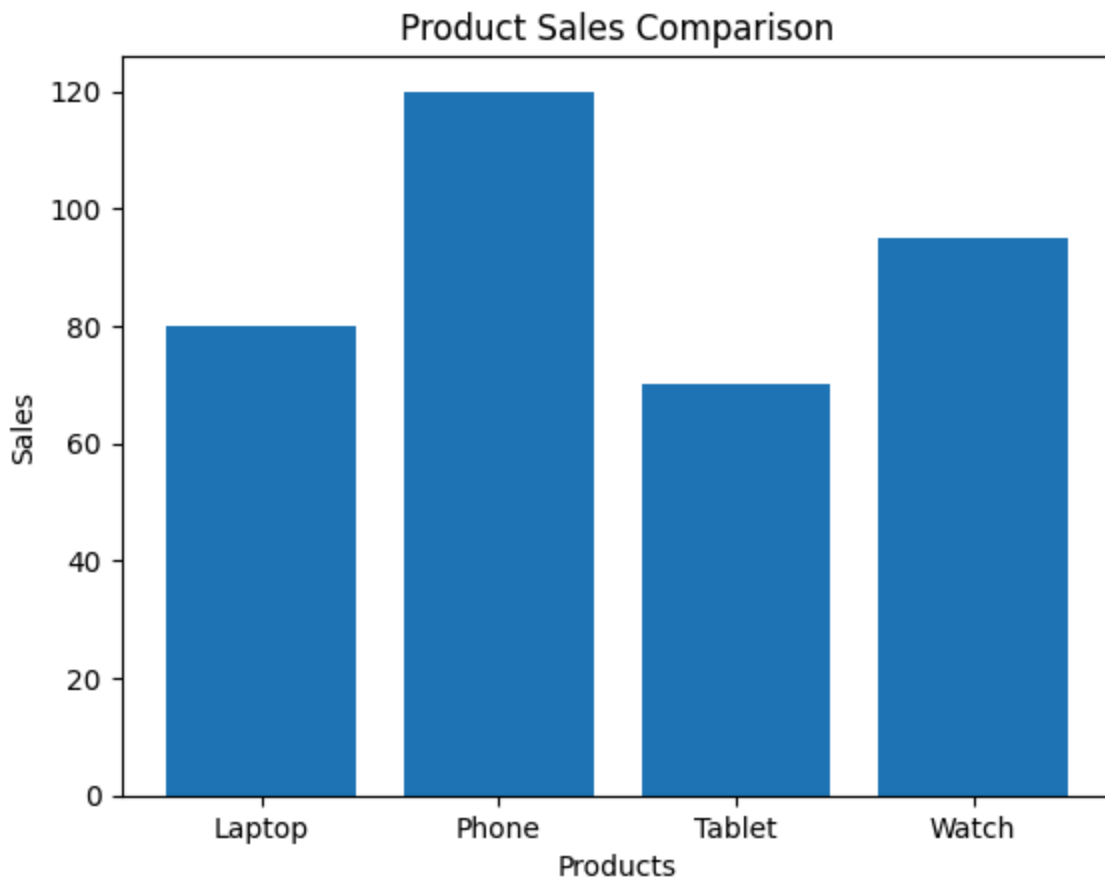


## ✓ 2.6 Bar Chart - Practical Example

A bar chart compares values across different categories using rectangular bars,

**Use case:** Comparing the sales performance of different products.

```
1 products=["Laptop","Phone","Tablet","Watch"]
2 sales=[80,120,70,95]
3
4 plt.bar(products,sales)
5 plt.title("Product Sales Comparison")
6 plt.xlabel("Products")
7 plt.ylabel("Sales")
8 plt.show()
```

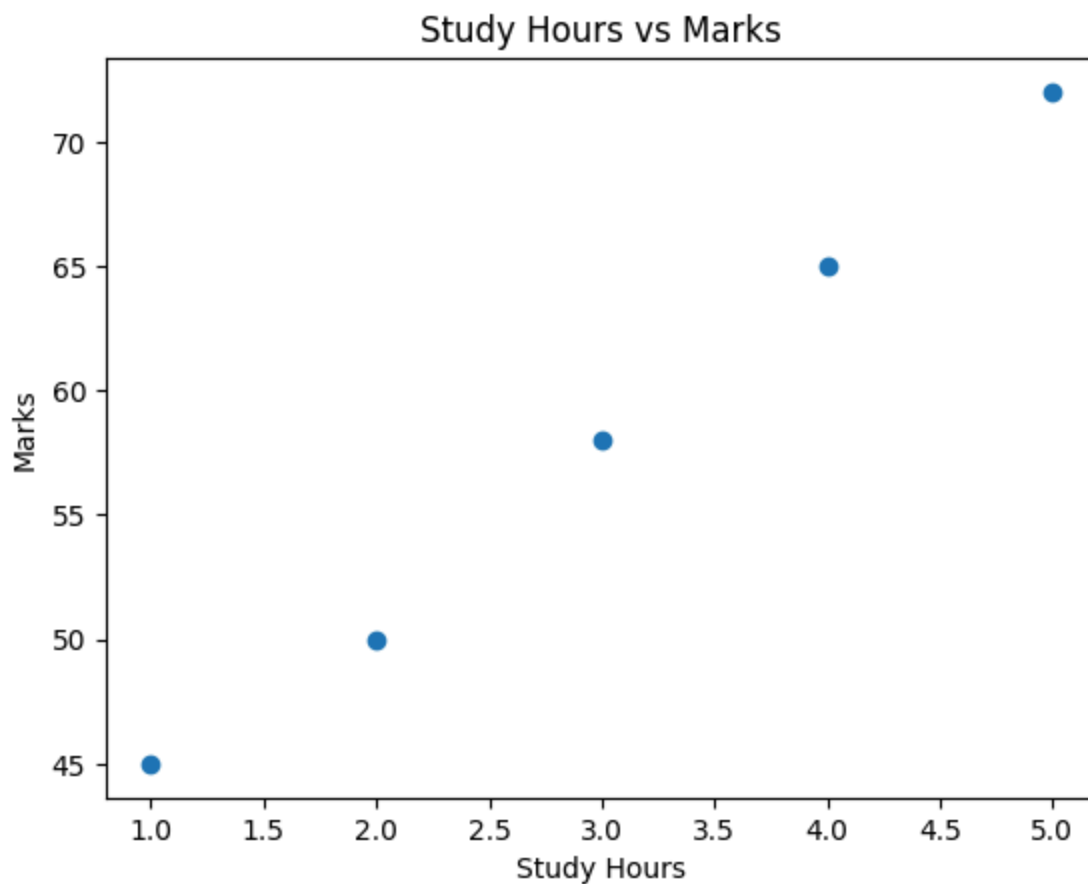


## ✓ 2.7 Scatter Plot - Practical Example

A scatter plot displays individual data points to show the relationship between two numerical variables.

**Use case:** Examining whether study time is related to students' marks.

```
1 study_hours=[1,2,3,4,5]
2 marks=[45,50,58,65,72]
3
4 plt.scatter(study_hours,marks)
5 plt.title("Study Hours vs Marks")
6 plt.xlabel("Study Hours")
7 plt.ylabel("Marks")
8 plt.show()
```

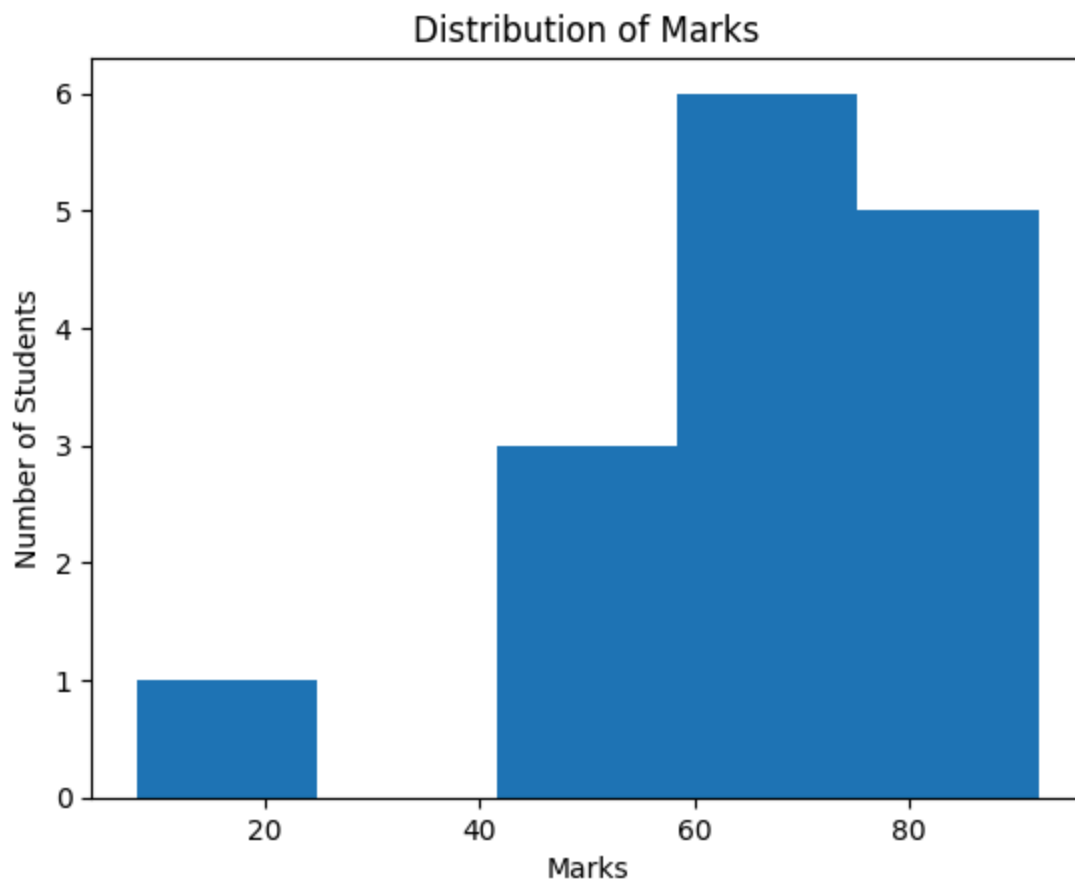


## ✓ 2.8 Histogram – Practical Example

A histogram shows how numerical values are distributed by grouping them into intervals.

**Use case:** Understanding the distribution of students' marks.

```
1 marks=[45,52,58,61,65,67,70,72,75,8,80,82,85,88,92]
2 plt.hist(marks,bins=5)
3 plt.title("Distribution of Marks")
4 plt.xlabel("Marks")
5 plt.ylabel("Number of Students")
6 plt.show()
```



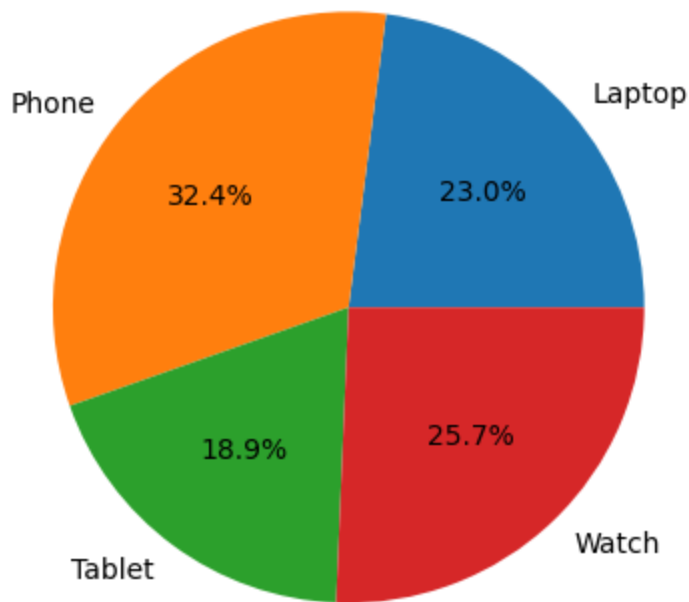
## ✓ 2.9 Pie Chart – Practical Example

A pie chart shows how different categories contribute to a complete total.

**Use case:** Showing the proportion of sales contributed by different products.

```
1 products=["Laptop","Phone","Tablet","Watch"]
2 sales=[85,120,70,95]
3
4 plt.pie(sales,labels=products,autopct="%1.1f%%")
5 plt.title("Sales Contribution by Product")
6 plt.show()
```

### Sales Contribution by Product

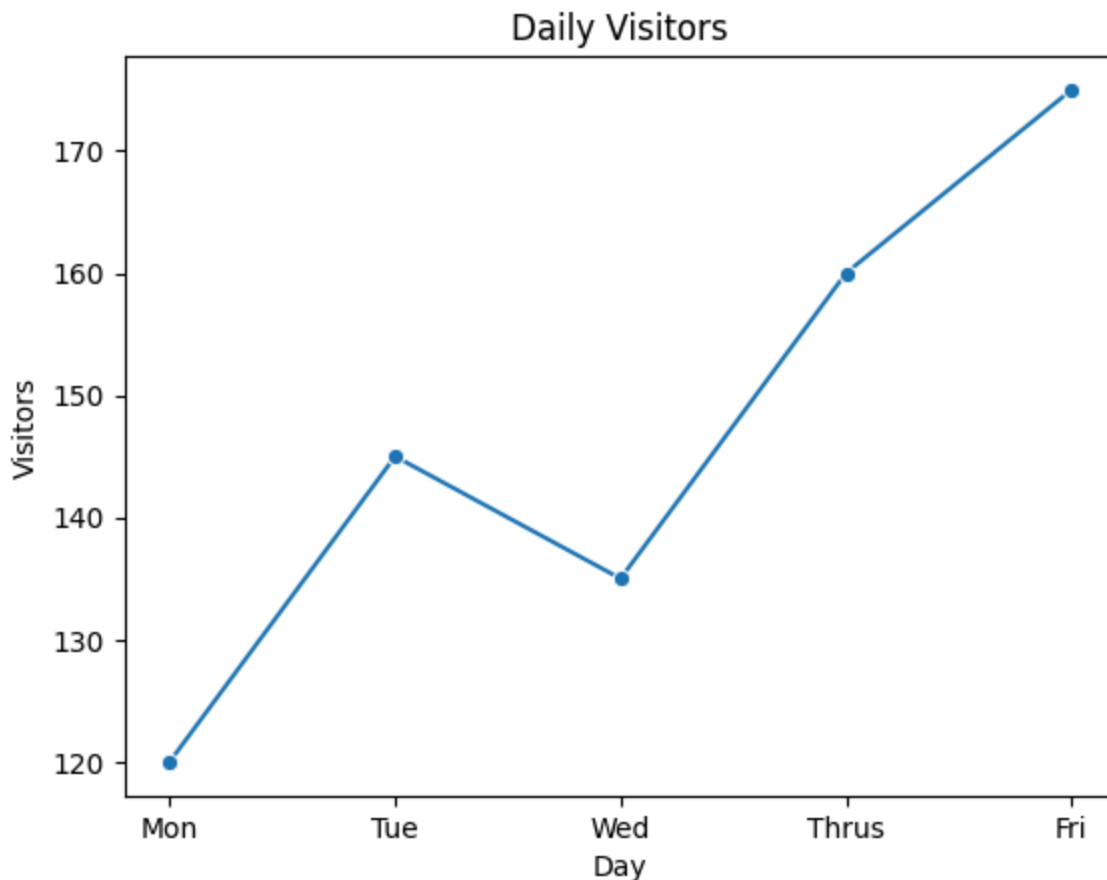


## ✓ 3. Seaborn – Common Graph Types

Seaborn provides a high-level interface for creating statistical graphics. It works closely with pandas DataFrames and offers convenient functions for exploring relationships and distributions in data.

The following examples demonstrate some commonly used Seaborn plots.

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 days=["Mon","Tue","Wed","Thrus","Fri"]
5 visitors=[120,145,135,160,175]
6
7 sns.lineplot(x=days,y=visitors,marker="o")
8 plt.title("Daily Visitors")
9 plt.xlabel("Day")
10 plt.ylabel("Visitors")
11 plt.show()
```

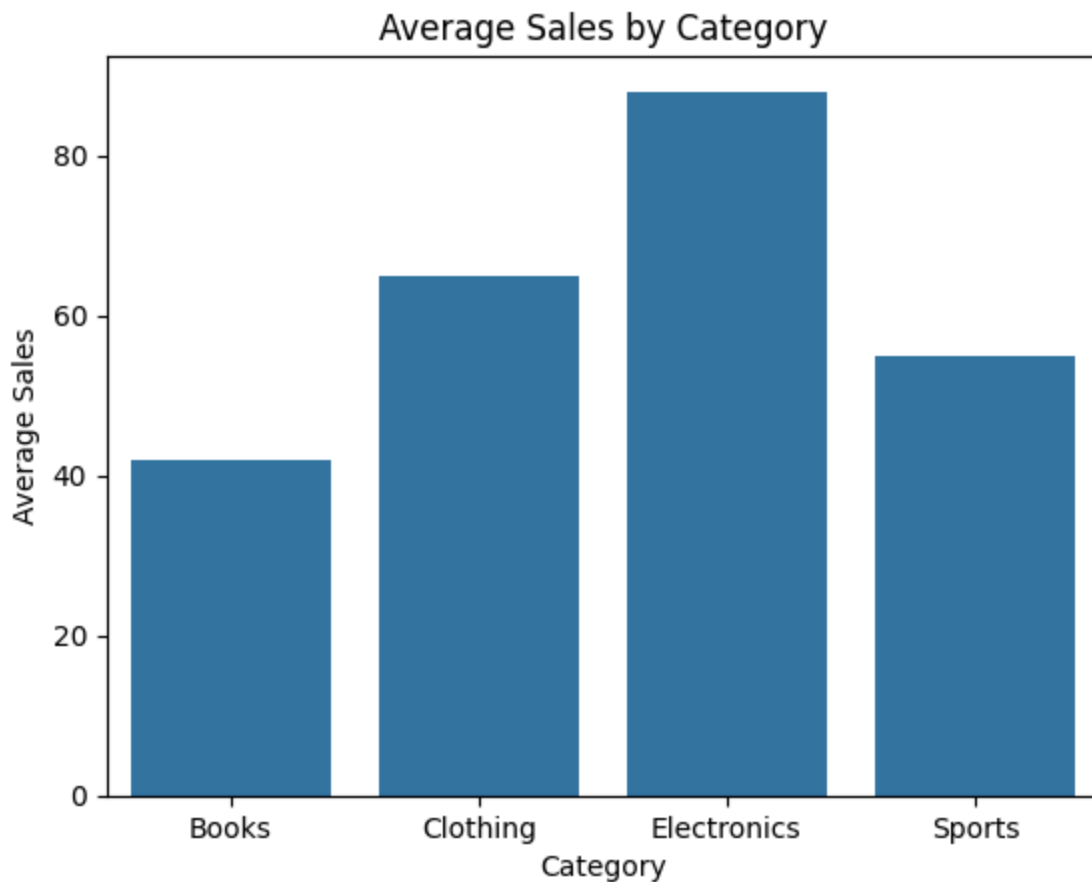


### ✓ 3.1 Bar Plot – Practical Example

A Seaborn bar plot compares a numerical value across different categories.

**Use case:** Comparing the average sales of different product categories.

```
1 categories = ["Books", "Clothing", "Electronics", "Sports"]
2 average_sales = [42, 65, 88, 55]
3
4 sns.barplot(x=categories, y=average_sales)
5 plt.title("Average Sales by Category")
6 plt.xlabel("Category")
7 plt.ylabel("Average Sales")
8 plt.show()
```

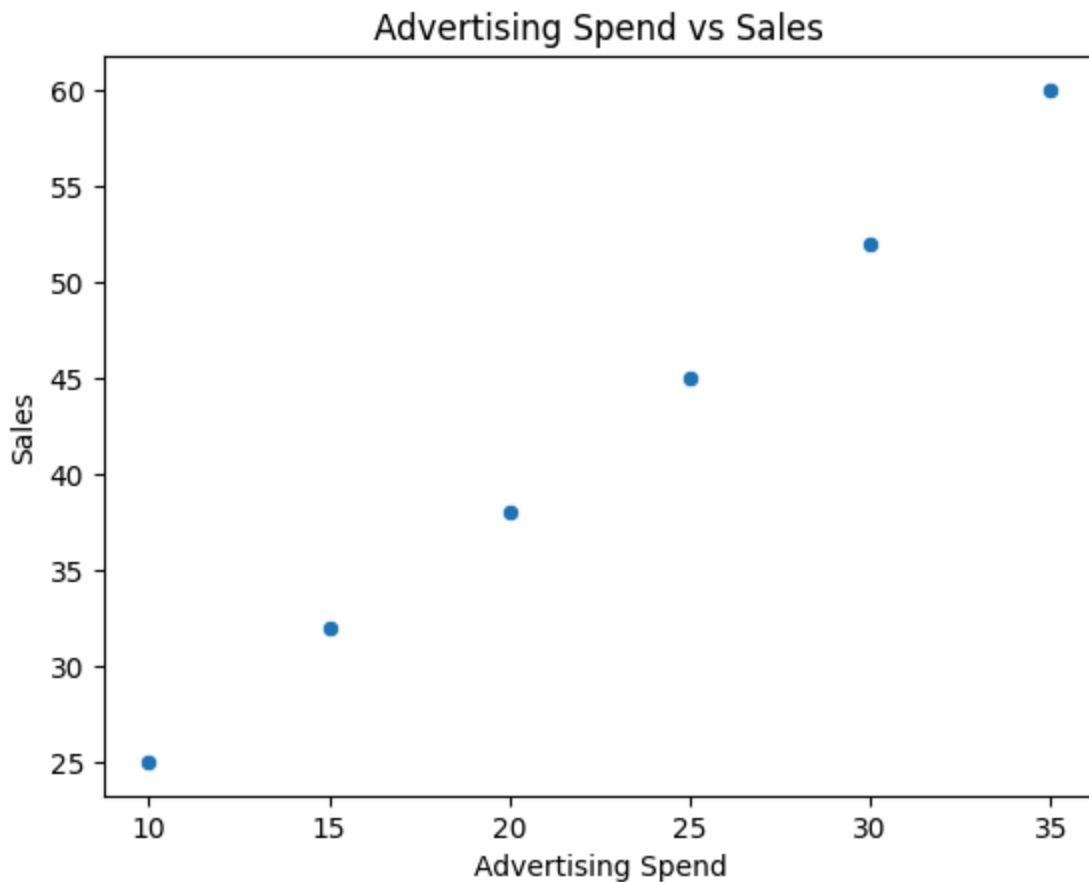


### ✓ 3.2 Scatter Plot – Practical Example

A Seaborn scatter plot displays individual observations and helps identify relationships between numerical variables.

**Use case:** Examining the relationship between advertising spending and sales.

```
1 ad_spend = [10, 15, 20, 25, 30, 35]
2 sales = [25, 32, 38, 45, 52, 60]
3
4 sns.scatterplot(x=ad_spend, y=sales)
5 plt.title("Advertising Spend vs Sales")
6 plt.xlabel("Advertising Spend")
7 plt.ylabel("Sales")
8 plt.show()
```

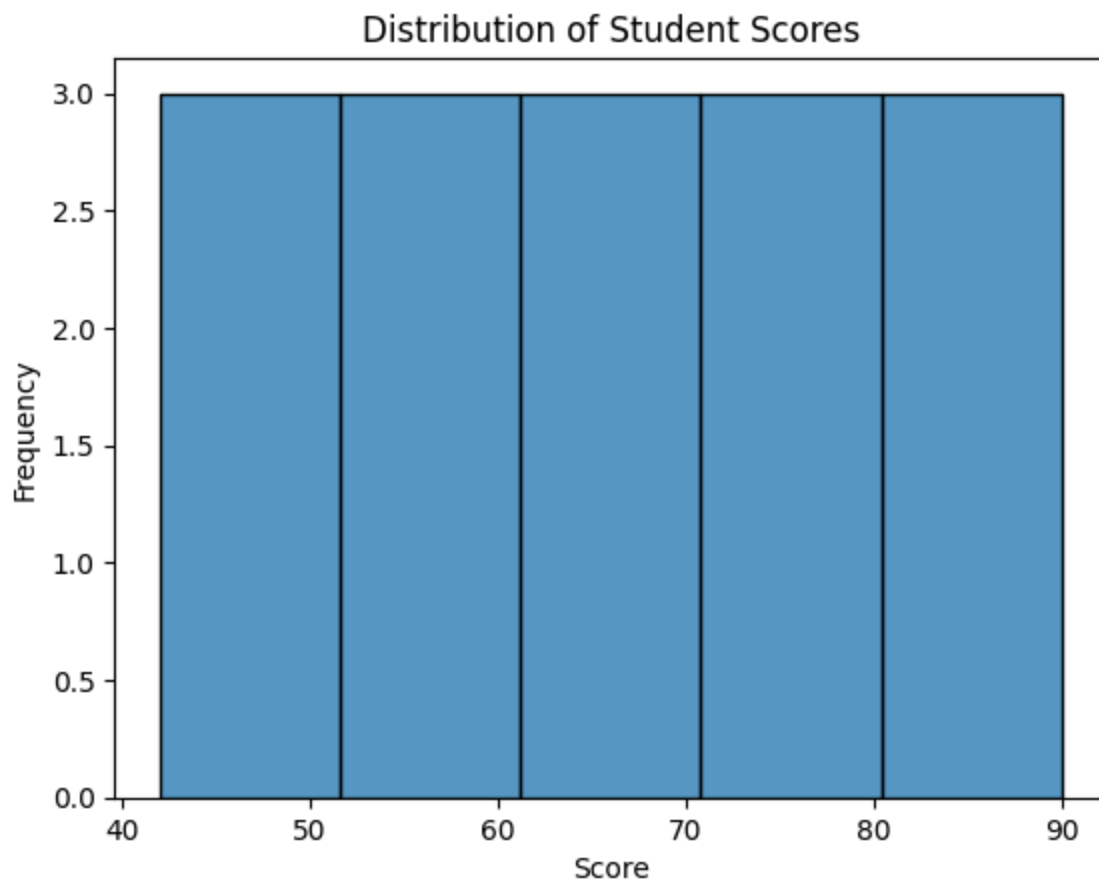


### ✓ 3.3 Histogram – Practical Example

A Seaborn histogram displays the distribution of numerical data by grouping values into intervals.

**Use case:** Analyzing the distribution of students' scores.

```
1 scores = [42, 48, 51, 55, 58, 61, 64, 67, 70, 72, 75, 78, 82, 86, 90]
2
3 sns.histplot(scores, bins=5)
4 plt.title("Distribution of Student Scores")
5 plt.xlabel("Score")
6 plt.ylabel("Frequency")
7 plt.show()
```

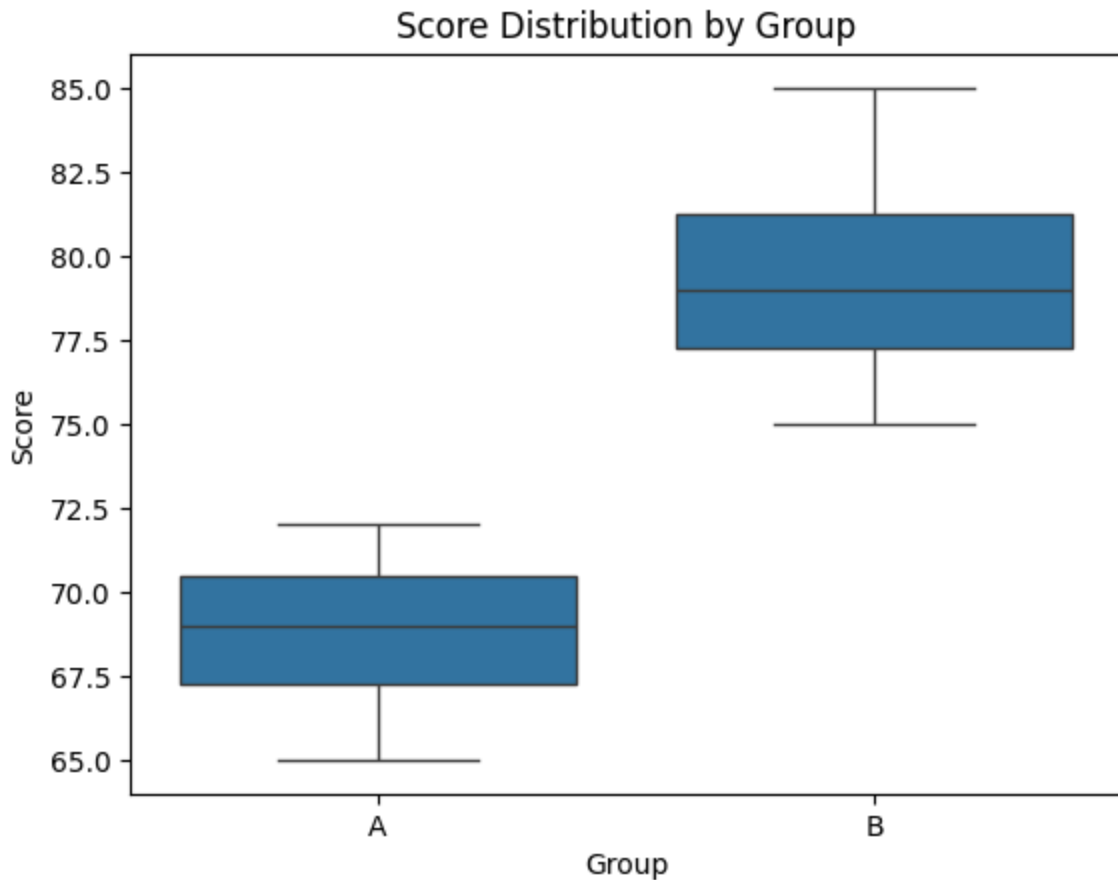


### ✓ 3.4 Box Plot – Practical Example

A box plot summarizes the distribution of numerical data and helps identify the spread and possible outliers.

**Use case:** Comparing the distribution of scores across different groups.

```
1 groups = ["A", "A", "A", "A", "B", "B", "B", "B"]
2 scores = [65, 70, 72, 68, 75, 80, 78, 85]
3
4 sns.boxplot(x=groups, y=scores)
5 plt.title("Score Distribution by Group")
6 plt.xlabel("Group")
7 plt.ylabel("Score")
8 plt.show()
```



### ✓ 3.5 Heatmap – Practical Example

A heatmap represents numerical values using different levels of shading, making patterns easier to identify.

**Use case:** Visualizing correlations between different numerical variables.

```
1 import pandas as pd
2
3 data = pd.DataFrame({
4     "Math": [80, 75, 90, 65],
5     "Science": [85, 70, 88, 72],
6     "English": [78, 82, 85, 68]
7 })
8
9 correlation = data.corr()
10
11 sns.heatmap(correlation, annot=True)
12 plt.title("Subject Score Correlation")
13 plt.show()
```

